



SIGNAL SERIES • OBSERVABILITY ON KUBERNETES

Observability Study Guide

The complete merged edition — **Loki + Promtail** for logs, **Prometheus + exporters** for metrics, and **Grafana** for visualization — deployed on **Minikube with Helm**. Fundamentals → architecture → deployment → LogQL/PromQL → app integration → troubleshooting → mental model → terms glossary → interview prep. One concrete lab: a Minikube cluster, a dedicated `observability` namespace, two Helm charts.

By **Nabawy**

AUTHOR & COMPILER

MERGED EDITION

Study + Deployment · A4

Loki

+ Promtail 

Prometheus

+ Exporters 

Grafana

Dashboards 

Minikube

+ Helm 



Table of Contents

	The Map — Executive Summary	3	08	Deployment 1/2 — Helm + prometheus-stack	15
01	What is Observability? — Four Pillars	4	08·b	Deployment 2/2 — loki-stack + The Gotcha	16
02	Logs vs Metrics vs Traces	5	09	Querying 1/2 — LogQL	17
02·b	The Core Stack at a Glance	6	09·b	Querying 2/2 — PromQL	18
03	Logs 1/3 — Why Loki?	7	10	Application Integration	19
03·b	Logs 2/3 — Loki Architecture	8	11	Troubleshooting Playbook 1/2	20
03·c	Logs 3/3 — Promtail Workflow	9	11·b	Troubleshooting 2/2 + Cleanup	21
04	Metrics 1/2 — Prometheus Fundamentals ...	10	13	The Mental Model — Architecture	22
04·b	Metrics 2/2 — The Metric Sources	11	13·b	Quick Revision — Terms Glossary	23
05	Traces — Journey of a Request	12	14	Interview Prep — 14 Concepts	24
06	Visualization — Grafana	13	14·b	Command Cheat Sheet	25
07	How It All Fits Together in K8s	14	14·c	Self-Quiz + Answer Key	26

■ **TASK [Status Color Legend]** theme guide

- TEAL — headings & series structure
 INDIGO — core concepts (KEY IDEA)
 GREEN — confirmations & proofs
 AMBER — cautions & notes
 RED — symptoms & failures

NOTE How to use this book: 14 connected parts merging the study guide and the deployment guide into one journey — defining Observability, the three signal types, Loki/Promtail architecture, Prometheus/exporters, Grafana, the full Minikube + Helm deployment, LogQL/PromQL querying, app integration, a troubleshooting playbook, a terms glossary, and interview prep. Every term stays in English exactly as used in the industry.

KEY IDEA The lab this book documents: one Minikube cluster · one dedicated `observability` namespace · two Helm charts (`kube-prometheus-stack` + `loki-stack`) · one shared label model gluing it all together.



The Map — The Whole Route

Before diving into any tool, here's the whole story — every sentence is a link to its chapter.

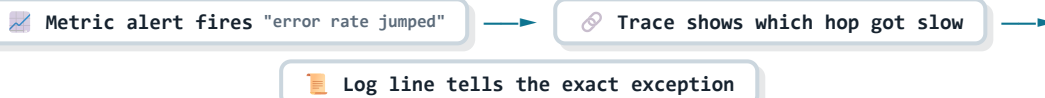
We begin in [Part 1 — What is Observability?](#): the ability to understand what's happening inside a system from the data it produces — **without shipping new code to ask a new question** — through four pillars: **Logs**, **Metrics**, **Traces**, and **Visualization**. Then [Part 2](#) puts the three signal types side by side — the single most exam-worthy table in this book — and [Part 2-b](#) introduces the deployed stack component by component.

Logs first: [Part 3 — Why Loki?](#) (10–20× cheaper — it indexes labels, not text), [Part 3-b](#) the write/read path pipeline (Distributor → Ingester → Store ← Querier), and [Part 3-c](#) Promtail's DaemonSet workflow: Discover → Tail → Label → Ship.

Metrics: [Part 4 — Prometheus](#) (pull model, time series, PromQL, service discovery) and [Part 4-b](#) the four metric sources: node-exporter, kube-state-metrics, cAdvisor, ServiceMonitor/PodMonitor CRDs. Then [Part 5 — Traces](#) (the natural next step) and [Part 6 — Grafana](#), the memoryless unified UI.

Everything practical: [Part 7](#) the complete data flow · [Part 8](#) the Helm deployment · [Part 8-b](#) the datasource conflict gotcha + port-forward · [Part 9](#) LogQL & [PromQL](#) · [Part 10](#) app integration · [Part 11](#) the troubleshooting playbook · [cleanup](#) · [Part 13](#) the architecture mental model · [the terms glossary](#) · [Part 14](#) interview prep, [cheat sheet](#) & [quiz](#). The route is clear. ►

▪ TASK [The classic debugging flow] how the signals cooperate



Grafana is the pane of glass where all of this comes together — because Loki uses **the exact same label model as Prometheus and Kubernetes**, you can pivot from a metric on a pod straight to that pod's logs.

KEY IDEA **The unifying idea:** each component does exactly one job — Promtail **pushes** logs, Prometheus **pulls** metrics, Grafana **queries both and displays them**. Two transport models, one shared label model (namespace/pod/container/node). No shared databases, no unnecessary complexity.



What is Observability? — The Four Pillars

Observability is the ability to understand what is happening inside a system by examining the data it produces — **without having to ship new code to ask a new question**. It relies on four core pillars:

■ **TASK [The Four Pillars]** and the question each answers



Logs

"What happened, and what did it say?"

Discrete, timestamped records of events — the error, the warning, the startup message.



Metrics

"How much, how fast, how often?"

Numeric measurements over time — CPU, requests per second, response latency.



Traces

"Where did the time go?"

The full path a single request takes as it moves across the different services.



Visualization

"Can a human act on this?"

Turns raw signals into dashboards a human can read and act upon.

The first three pillars produce and store the signals — the fourth makes them understandable and actionable.

KEY IDEA The pillars don't overlap — each answers a **different kind of question**: a **Log** tells you *what happened in detail*, a **Metric** tells you *how much and how fast*, a **Trace** tells you *where the time went*. A full investigation uses all three together.

NOTE · Why the definition matters Notice the key phrase: "without shipping new code to ask a new question." You instrument the system **once**; after that, any new question ("why was latency high at 2 AM?", "which pod threw the error?") is answered by querying data that **already exists** — not by redeploying with extra print statements. A good observability system answers questions you didn't anticipate when you built it — unlike traditional monitoring, which only answers pre-written questions.

EXAM The classic interview opener: "What is Observability, and what are its pillars?" → **Understanding system state from its data without deploying new code** · Pillars: **Logs · Metrics · Traces · Visualization** — with the question each one answers.

▶▶ How do the pillars become actual components? → [Part 2: Logs vs Metrics vs Traces](#)



Logs vs Metrics vs Traces

The three signal types are **complementary, not redundant**. This table is the single most exam-worthy artifact in this guide:

	Logs	Metrics	Traces
Question	What happened, and what did it say?	How much, how fast, how often?	Where did the time go?
Nature	Discrete, timestamped event records	Numeric measurements over time	Full journey of one request across services
Shape	Free-form text lines	Time series (name + labels + values)	Request path through service hops
Stored by	Loki	Prometheus	<i>Not deployed in this lab</i> (Tempo / Jaeger / Zipkin would be)
Query language	LogQL	PromQL	(trace UI)
Best for	Reading the <i>exact</i> content of an event ("show me the stack trace")	Trends, rates, aggregation, alerting, dashboards	Latency breakdown across a multi-service request

■ TASK [How they work together] the classic debugging flow



Grafana is the pane of glass where all of this comes together.

KEY IDEA Because Loki uses **the exact same label model as Prometheus and Kubernetes**, you can pivot from a metric on a pod straight to that pod's logs. See a CPU spike on `pod X, namespace Y` in a Prometheus panel → query `{namespace="Y", pod="X"}` in Loki → read the logs from exactly that window. **Zero extra configuration produces this.**

EXAM Logs = **what happened** (Loki/LogQL) · Metrics = **how much/how fast** (Prometheus/PromQL) · Traces = **where the time went** (not in this lab) · Debug flow: **alert** → **trace** → **log**, all visible in Grafana.

▶ Now meet the components that store these signals → [Part 2-b: The Core Stack](#)

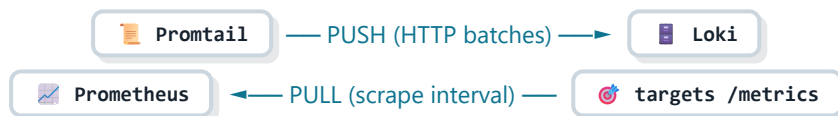


The Core Stack — At a Glance

TASK [Seven components, seven jobs] who does what

Component	Role	Installed by	Runs as
Loki	Stores and indexes logs	loki-stack	Log backend
Promtail	Tails container logs on every node and ships them to Loki	loki-stack	DaemonSet
Prometheus	Scrapes and stores metrics as time series	kube-prometheus-stack	Metrics backend
node-exporter	Exposes host-level metrics	kube-prometheus-stack	DaemonSet
kube-state-metrics	Exposes Kubernetes object state metrics	kube-prometheus-stack	Deployment
Alertmanager	Routes PromQL alert conditions to Slack/email/PagerDuty	kube-prometheus-stack	DISABLED IN THIS LAB
Grafana	Queries both backends, renders dashboards	kube-prometheus-stack	Deployment

TASK [Two transports, one label model] the asymmetry



KEY IDEA **Key asymmetry to remember:** Promtail **pushes** logs to Loki; Prometheus **pulls** metrics from targets. Two different transport models — **one shared label model** (namespace / pod / container / node) that makes the stack feel like a single system.

KEY IDEA • Modularity These are **three fully independent groups** — you could **swap any one for an alternative** (ELK instead of Loki, for example) because they don't talk to each other directly. They only meet at Grafana. **That separation is why the stack stays flexible.**

WATCH OUT Grafana **stores nothing** — it's display only. Any dashboard without live Data Sources = an empty page. The data always lives in Loki or Prometheus. And note that Grafana itself is installed **by kube-prometheus-stack**, not standalone — this exact fact causes the datasource conflict in [Part 8-b](#).

EXAM Stack: **Loki + Promtail (Logs) • Prometheus + exporters (Metrics) • Grafana (Visualization)** • Push = logs, Pull = metrics • Promtail & node-exporter run as **DaemonSets** • Alertmanager disabled in the lab • Components are swappable — they meet only at Grafana.

Starting with the first pillar: why Loki specifically? → [Part 3](#)

Why Loki? — The Philosophy & the Differences

Loki is a horizontally-scalable, multi-tenant log aggregation system built by **Grafana Labs**. It stands apart from full-text systems like ELK for four essential reasons:

💰 Cost-Effective — the cheapest

10–20× cheaper for storage, because Loki indexes **only metadata (labels)** — not the full text of every log line. Light indexing = light storage bill.

🗄️ Storage — cheap tiers

Logs are compressed into **chunks** and stored in **cheap object storage** (or a local filesystem in labs) — no massive search nodes needed.

🔍 Querying — LogQL

You query Loki with **LogQL** — a language **deliberately designed** to be similar to PromQL. Learn one and the other feels familiar.

🔗 Integration — native fit

Native **Grafana integration** (no separate UI needed), and it uses the **exact same label model as Kubernetes and Prometheus**.

▪ TASK [Loki vs ELK — the difference in one line] the trade-off

ELK: indexes full text → powerful text search 🍌



Loki: indexes labels only → 10–20× cheaper 💰

KEY IDEA • The mental model + the philosophy Think of Loki as "**Prometheus for logs**." And the philosophy in one line: **"don't index what you won't query."** Indexing metadata (which namespace? which pod? which container?) is enough to filter millions of lines, leaving the raw text to be read only at query time — that's the root cause of both its cheapness and its query behavior: **label selectors are fast, line filters are done on the fly**.

WATCH OUT The trade-off is real: because Loki doesn't index the log text itself, free-form full-text search is weaker than ELK. You query **by labels first**, then narrow with LogQL. In Kubernetes environments with good labels — exactly our case — the trade-off pays off enormously.

🏠 **EXAM** The four reasons: **Cost (10–20× cheaper — labels-only indexing) • Storage (chunks in cheap object storage) • LogQL (similar to PromQL) • native Grafana integration + same label model**. Mental model: **"Prometheus for logs"** • Philosophy: **"don't index what you won't query."**

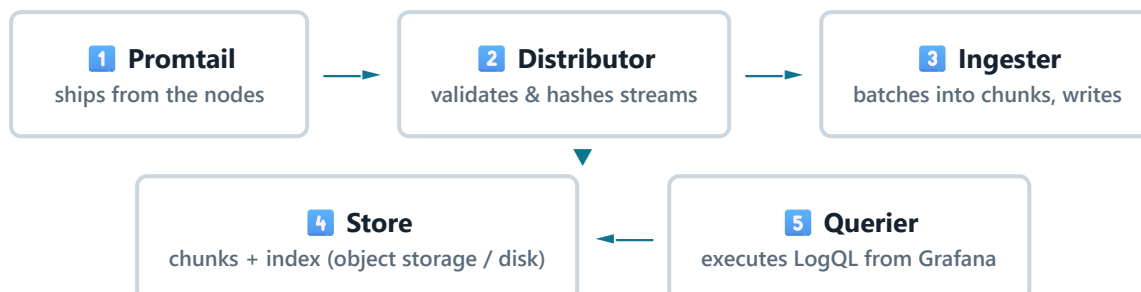
🔗 How does Loki work internally? The write and read paths: → [Part 3-b](#)



Loki Architecture — Write Path & Read Path

3-A The five-stage pipeline

Logs travel through Loki's architecture along a specific path — writes flow down the pipeline, reads come back from the Store:



WRITE: container log files → Promtail → Distributor → Ingestor → Store · READ: Grafana —LogQL→ Querier → reads chunks + index from Store/Ingestor

Component	Job
1. Promtail	Ships logs from the nodes
2. Distributor	Validates and hashes the incoming log streams
3. Ingestor	Batches the streams into chunks and writes them
4. Store	Stores the chunks and the index (object storage or local disk)
5. Querier	Executes the LogQL queries coming from Grafana

KEY IDEA A **log stream** = a **unique combination of labels**. Every distinct label set is its own stream — which is exactly why the labels Promtail attaches matter so much ([Part 3:c](#)).

NOTE • Reads vs writes Writes flow left→right down the pipeline (1→4). Reads never touch the write path's front: the **Querier** (5) pulls already-stored chunks + index from the Store (and Ingestor for recent, not-yet-flushed data).

EXAM The pipeline: **Promtail** → **Distributor (validates & hashes)** → **Ingestor (chunks)** → **Store (chunks + index)** ← **Querier (LogQL)**. A log stream = a **unique label combination**.

▶▶ The component that feeds stage 1 — why it's a DaemonSet: → [Part 3:c: Promtail](#)

Promtail Workflow — Why a DaemonSet + the 4 Steps

Promtail runs as a **DaemonSet** — **one pod per Kubernetes node**. Why? Because container logs are **files on each node's local filesystem**. A Promtail pod on node A physically cannot read node B's `/var/log/pods`. **Node-local data ⇒ node-local collector**.

▪ TASK [The four steps] discover → tail → label → ship

1 Discover

Uses Kubernetes **service discovery** to find every pod running **on its node**.

2 Tail

Reads container log files directly from `/var/log/pods` — like a `tail -f`.

3 Label

Attaches K8s metadata as labels: **namespace, pod, container, node**.

4 Ship

Batches log lines and **pushes to Loki's distributor over HTTP**.

KEY IDEA • Labels are the gear in the machine Step 3 matters more than it looks: because Promtail's labels are **identical to Prometheus's labels**, switching between logs and metrics in Grafana is seamless — **same namespace, same pod, same language** — and logs and metrics are **cross-correlatable by default**. See a CPU spike on `pod X, namespace Y` in a Prometheus panel → query `{namespace="Y", pod="X"}` in Loki → read the logs from exactly that window. **Zero extra configuration produces this.**

WATCH OUT • Label cardinality Labels are indexed, so they're relatively expensive in Loki — never attach unbounded varying values (like `user_id`) as labels. Keep it to stable, structured ones: **namespace, app, pod, container, node**.

NOTE Step 4 completes the push model: Promtail **pushes** batches over HTTP — the opposite of Prometheus, which **pulls** from targets. Same labels, opposite transport direction.

 **EXAM** Promtail = **DaemonSet** (one pod per node — logs are node-local files in `/var/log/pods`) · Steps: **Discover → Tail → Label (same K8s labels as Prometheus) → Ship (HTTP push)** · Shared labels = free log↔metric correlation.

 From logs to metrics: → [Part 4: Prometheus](#)



Prometheus — Fundamentals

Prometheus is an open-source monitoring system that uses a **pull model**: it **scrapes** metrics from targets on a regular interval, rather than waiting for applications to push data.

■ TASK [The three core concepts] storage · querying · alerting

📄 Storage

Every measurement is a **Time Series**:
a **metric name** + **key/value labels** +
a **sequence of timestamped values**.

Example:

```
container_cpu_usage_seconds_total{name="api-7d9f"}  
pod="api-7d9f"} — labels are the  
dimensions you aggregate by.
```

🔍 Querying — PromQL

A powerful query language for
aggregating, **filtering**, and
computing **rates** over time series.

🔔 Alerting — Alertmanager

Pairs with **Alertmanager** to turn
PromQL conditions into **routed
alerts** — Slack, Email, PagerDuty.
(Disabled in this lab — but the concept
stands.)

■ TASK [Why pulling matters] practical consequences

CENTRAL CONTROL The Pull Model gives Prometheus full control: it knows **when** the last scrape happened and decides the cadence and the target list — apps don't need to know where Prometheus lives. Just expose a `/metrics` endpoint, nothing more.

FAILURE IS VISIBLE A dead target doesn't just leave a gap in the data — its `up` metric goes to `0`. A target that won't respond to a scrape = down. You can literally **query whether monitoring works**.

■ TASK [Service discovery] no manual IP lists

In Kubernetes, Prometheus automatically finds targets via service discovery + Custom Resources (CRDs)

Nobody maintains a list of pod IPs — pods are ephemeral anyway; the list would be stale in minutes.

WATCH OUT Every unique combination of labels creates a **new series** — the same label-explosion issue as Loki: too many varying labels = an explosion in series count.

🎓 **EXAM** Prometheus: **Pull Model** · Time series = **name + labels + timestamped values** · Querying: **PromQL** · Alerting: **Alertmanager** (disabled here) · Target discovery: **automatic via service discovery + CRDs** · Dead target = `up == 0`.

➡ Where do the metrics actually come from? → [Part 4-b: The Metric Sources](#)

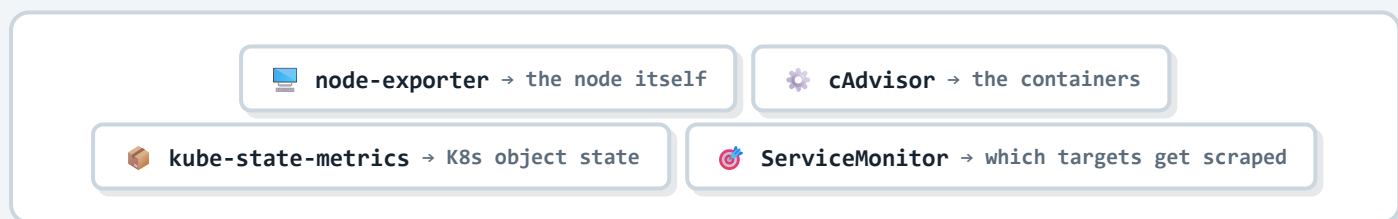


Metric Sources — Know Who Exposes What

Component	Where it runs	What it exposes	The question it answers
node-exporter	DaemonSet (one pod per node)	Host-level metrics: CPU, memory, disk, network	"Is the machine itself healthy?"
kube-state-metrics	Watches the Kubernetes API	Object state : pod status, deployment replicas, restarts	"Does Kubernetes <i>think</i> everything is fine?"
cAdvisor	Built into the kubelet	Per-container resource usage (actual CPU/memory each container consumes)	"Which container is eating resources?"
ServiceMonitor / PodMonitor	CRDs — not pods at all	Declarative rules for which Services/Pods Prometheus should scrape	"How does Prometheus find <i>my app</i> ?"

KEY IDEA • Drill this distinction node-exporter = **machine** · kube-state-metrics = **Kubernetes objects** · cAdvisor = **containers**. Three layers of "is it healthy" — three different components. And the ServiceMonitor CRD is what wires your app into Prometheus with zero manual config.

■ TASK [Who watches what?] the layering



NOTE • Why CRDs matter Instead of asking "which IPs do I scrape?" — you create a ServiceMonitor naming the Service by label, and the Helm chart that deploys the app ships its ServiceMonitor along. The result: **deploying an app = monitoring it automatically**. Deploying the CRD is the configuration. (Full example in [Part 10](#).)

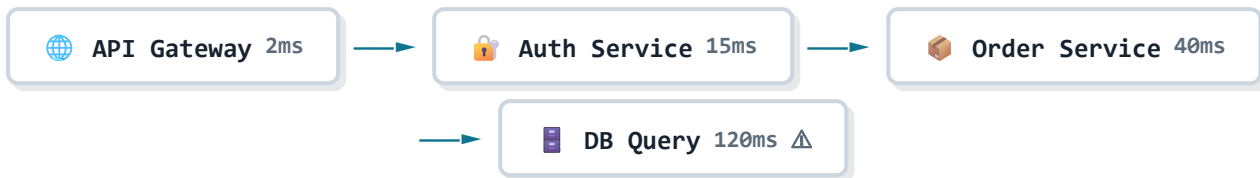
EXAM **node-exporter** (DaemonSet — host metrics) · **kube-state-metrics** (API server — object state) · **cAdvisor** (in the kubelet — per-container) · **ServiceMonitor/PodMonitor CRDs** (declarative scrape targeting, auto-wired by Helm).

▶▶ The third pillar: the journey of a single request → [Part 5: Traces](#)

Traces — The Journey of a Request

A **trace** records the full journey of a **single request** as it moves through multiple services:

▪ **TASK [The journey of one request]** a worked example



The trace reveals instantly: **most of the time went to the DB Query** — no guessing, no manually checking every service.

5.1 The purpose

Traces show **exactly where time was spent** during a request. Metrics tell you *that* requests are slow; traces tell you *which hop* made them slow — which service was slow, which call repeated, where the real bottleneck lives.

5.2 The tools

Tracing typically uses **Grafana Tempo**, **Jaeger**, or **Zipkin**, with applications instrumented via **OpenTelemetry**.

NOTE · In this lab **Traces are not deployed in this specific lab** — but they represent **the natural next step** once logs (Loki) and metrics (Prometheus) are in place. Two of the three signals are already running.

KEY IDEA When someone tells you "the API is slow" — Logs tell you *what happened*, Metrics tell you *how slow*, but only the **Trace** tells you **exactly where** inside the request journey the time went.

EXAM A Trace = **the journey of one request across multiple services** · Purpose: **where the time was spent** · Tools: **Grafana Tempo / Jaeger / Zipkin** via **OpenTelemetry** · **not deployed in this lab** — the next step.

▶ The fourth pillar: the window into everything → [Part 6: Grafana](#)

Grafana — The Unified UI

Grafana is the unified UI for observability — the pane of glass where logs and metrics meet.

TASK [Four concepts] data · config · dashboards · explore

🔧 Data Handling

Grafana **does not store data itself** — it connects to **Data Sources** (Prometheus, Loki) and **queries them live** on every panel refresh. Wipe the backends and Grafana has nothing to show — **it's a lens, not a database**.

⚙️ Configuration

Each Data Source requires: a **URL** + **authentication** + the **query language** it should speak (**PromQL** or **LogQL**).

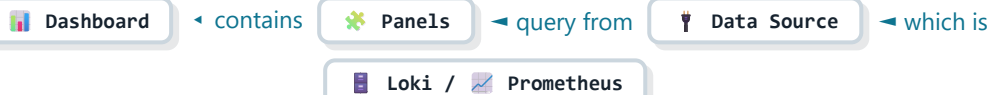
📊 Dashboards & Panels

A **Dashboard** is a **collection of Panels**. Each panel runs a query and renders it as a **graph, table, log stream, or gauge**. Dashboards are **JSON** — version-controlled, or imported from `grafana.com/dashboards` by numeric ID.

🔍 Explore Mode

Run **ad-hoc PromQL or LogQL** queries without building a dashboard — the go-to tool for debugging ("just show me the errors from that namespace *right now*").

TASK [The hierarchy inside Grafana] the hierarchy



KEY IDEA Grafana is deliberately memoryless: **every panel = a live query** at the moment of viewing. Nobody asks "is the dashboard up to date?" — it's up to date by definition, because it queries the source directly on every load.

✅ **EXPLOIT THIS · Dashboards as code** Dashboards are JSON — **commit them to Git**. A lost dashboard = **restore from Git**; an intentional change = a **Pull Request** reviewed like any other code. Version-controlling dashboards is a feature to exploit, not an accident.

WATCH OUT · Deployment detail In this stack, Grafana is **installed by** `kube-prometheus-stack`, not deployed standalone. That fact is **exactly what causes the datasource conflict** in [Part 8-b](#) — the single best "bug I debugged" story in this guide.

🎓 **EXAM** Grafana: **stores nothing — live Data Sources** · Data Source = **URL + Auth + Query Language** · Dashboard = set of Panels (graph/table/logs/gauge) · Dashboards = **JSON** → **Git: restore + PR review** · Explore = **ad-hoc debugging**.

🔗 All the pieces are ready — how do they fit together in Kubernetes? → [Part 7](#)

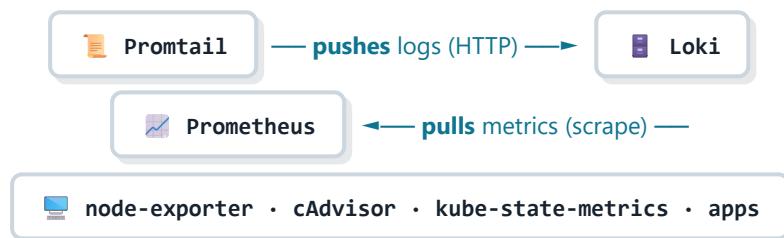


How It All Fits Together — in Kubernetes

TASK [The complete data flow] six steps, end to end

1. **Application pods** write logs and (optionally) expose `/metrics`.
2. On every node, **Promtail** (DaemonSet) tails `/var/log/pods`, labels each line with Kubernetes metadata, and **pushes** batches to Loki's distributor.
3. On every node, **node-exporter** (DaemonSet) exposes host metrics; **cAdvisor** (in the kubelet) exposes per-container metrics; **kube-state-metrics** exposes object state.
4. **Prometheus** **pulls** all of those metrics on a scrape interval, discovering targets via Kubernetes service discovery and CRDs (ServiceMonitor/PodMonitor).
5. **Grafana** holds both as data sources and **queries Prometheus (PromQL)** and **Loki (LogQL)** live.
6. You open Grafana in a browser through a **port-forward tunnel** and see everything.

TASK [Two transports, one label model] the glue



But both carry the **same labels** — namespace / pod / container / node — which is what makes the stack feel like one system.

KEY IDEA Logs are **pushed**; metrics are **pulled**. The transport directions differ, but the **label vocabulary is identical** — that shared label model is the glue. It's why one label pivot in Grafana jumps you from a CPU graph to the exact log lines that explain it.

NOTE · Access pattern Grafana is a `ClusterIP` Service — reachable only from *inside* the cluster. From your laptop you reach it via `kubectl port-forward` (full command + decode in [Part 8-b](#)).

EXAM Flow: **apps/nodes produce** → **Promtail pushes logs** → **Loki** · **Prometheus pulls metrics** → **TSDB** · **Grafana queries both live** → **browser via port-forward**. Push ≠ Pull, but labels are shared.

Time to actually deploy this — with two Helm commands: → [Part 8: Deployment](#)

Helm + kube-prometheus-stack

Helm is a **package manager for Kubernetes**. It bundles all necessary manifests — Deployments, Services, ConfigMaps, CRDs — into "**charts**." Installing a chart creates a **release**. Charts live in **repositories** you add to your host. Instead of dozens of hand-written YAML files, **two Helm commands** deploy the entire platform — including default dashboards and wired-up data sources. **Helm turned a week-long project into two commands.**

TASK [Setup — repos + namespace] best practice: dedicated namespace

```
# add both chart repositories, refresh indexes, create the namespace
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo add grafana https://grafana.github.io/helm-charts
helm repo update
kubectl create namespace observability # keeps monitoring separate from app workloads
```

TASK [Chart 1: kube-prometheus-stack] one chart, many components

INSTALLS Prometheus · Alertmanager · Grafana · node-exporter · kube-state-metrics · ServiceMonitor/PodMonitor CRDs · default Kubernetes dashboards — in one shot.

⚠ RESOURCE MANAGEMENT By default this chart **requests more CPU and RAM than a small Minikube cluster can handle** — install it bare and pods sit in `Pending` because the scheduler can't fit their requests anywhere. You must **drastically trim resource requests and limits for every component** with `--set` flags:

```
helm upgrade --install kube-prometheus-stack prometheus-community/kube-prometheus-stack \
  -n observability \
  --set alertmanager.enabled=false \
  --set prometheus.prometheusSpec.retention=6h \
  --set grafana.adminPassword=admin123 \
  --set prometheus.prometheusSpec.resources.requests.cpu=100m \
  --set prometheus.prometheusSpec.resources.requests.memory=256Mi \
  --set grafana.resources.requests.cpu=50m \
  --set grafana.resources.requests.memory=128Mi
# ...similar trimming for kube-state-metrics & node-exporter.
# Exact value keys: helm show values prometheus-community/kube-prometheus-stack
```

Override	Why
Resources trimmed (every component)	Chart defaults assume beefy nodes; Minikube pods go <code>Pending</code> otherwise
<code>retention=6h</code>	TSDB storage grows with retention; a lab needs no history — 6h is plenty for debugging
<code>alertmanager.enabled=false</code>	No alert routing needed in a lab → fewer pods, less RAM
<code>adminPassword=admin123</code>	The lab's Grafana credentials (<code>admin</code> / <code>admin123</code>)

🔑 IDEMPOTENCY RULE Always use `helm upgrade --install`, never plain `helm install`. `helm upgrade --install` is **safe to re-run**: if the release exists it upgrades in-place; if not, it installs. Plain `helm install` **fails** on a second run.

📖 EXAM Helm = **K8s package manager** (charts → releases, from repos) · **two commands = the whole platform** · Minikube needs **trimmed resource requests** or pods go `Pending` · **retention 6h** saves disk · **helm upgrade --install** = idempotent.

▶ Chart 2 — and the one non-obvious gotcha: → [Part 8-b](#)



loki-stack + The Gotcha + Port-Forward

```
helm upgrade --install loki-stack grafana/loki-stack \
  -n observability \
  --set grafana.sidecar.datasources.defaultDataSourceEnabled=false # ← THE critical flag
```

TASK [⚠ The datasource provisioning conflict] the one non-obvious gotcha

1. `loki-stack` ships **its own Grafana data-source ConfigMap** for Loki.
2. The **Grafana sidecar** (in the Grafana installed by `kube-prometheus-stack`) **auto-discovers** datasource ConfigMaps and loads them — Loki appears with zero manual config. Nice.
3. **The conflict:** `kube-prometheus-stack` marks **Prometheus** as the **default** datasource. `loki-stack` **also tries to mark Loki as default**.
4. Grafana allows **exactly one default datasource**. Two defaults = **"Datasource provisioning error"** → Grafana **crashes** → **restarts** → **crashes** → `CrashLoopBackOff`.

✓ **THE FIX** Install Loki with `--set grafana.sidecar.datasources.defaultDataSourceEnabled=false` — Loki still becomes a datasource, just **not the default** (Prometheus stays default).

🔧 **EMERGENCY REPAIR** Already broken? `kubectl get configmap -n observability` → find the extra datasource ConfigMap → **remove** `isDefault: true` → restart the Grafana pod.

TASK [Accessing Grafana — port-forward] ClusterIP = inside the cluster only

```
kubectl port-forward --address 0.0.0.0 -n observability \
  svc/kube-prometheus-stack-grafana 3000:80
# then open http://localhost:3000 → login admin / admin123
```

Piece	Meaning
<code>svc/kube-prometheus-stack-grafana</code>	The Service to tunnel to
<code>-n observability</code>	The namespace the Service lives in
<code>3000:80</code>	local port 3000 → Service port 80
<code>--address 0.0.0.0</code>	Bind on all host interfaces , not loopback only — required on Windows/WSL , otherwise <i>connection refused</i> from the Windows side

DASHBOARDS Pre-installed (by `kube-prometheus-stack`): cluster compute resources, namespaces, nodes, Kubelet — all Prometheus-backed. **Not included:** Loki dashboards — import via UI (**Dashboards** → **Import** → **ID** → **Load**): **13639** (Kubernetes Logs) · **12611** (Log volume) — or skip dashboards and use **Explore**.

📖 **EXAM** Two charts both provisioning a **default** datasource → Grafana crash-loops (it allows exactly one) → fix: `defaultDataSourceEnabled=false` · Grafana is **ClusterIP** → `port-forward` with `--address 0.0.0.0` on WSL → `admin/admin123` · Import dashboards **13639 / 12611**.

▶▶ The stack is up — now query it: → [Part 9: LogQL](#)

LogQL — Querying Loki

Framing: two backends, two query languages, one UI. LogQL was *deliberately designed to be similar to PromQL*, and both select by the *same labels* — learn one, and the other feels familiar.

- **TASK [Start in Explore mode]** no dashboard needed

Grafana → Explore → pick datasource → run ad-hoc queries

 **Loki:** {namespace="observability"} → live streams

 **Prometheus:** up → 1 = healthy, 0 = down

- **TASK [LogQL — every query has up to three stages]** selector → pipeline → wrapper

1 Stream selector

Labels — **indexed, fast**. Picks which log streams to read.

2 Filter pipeline

Line filters — evaluated at **query time** on the matched streams.

3 Metric wrapper

Turns log counts into **numbers** — logs become a metric.

```
# 1) Stream selector: all logs from a namespace
{namespace="observability"}
# narrow by pod / container (the labels Promtail attached)
{namespace="default", pod="my-app-7d9f8b6c-", container="app"}

# 2) Line filters: keep / drop lines by content (Loki searches text HERE)
{namespace="observability"} |= "error"
{namespace="observability"} |= "error" != "timeout"

# 3) Metric query: count matching log lines per pod over a 5m window
sum by (pod) (count_over_time({namespace="default"} |= "error" [5m]))
```

KEY IDEA • Why this ordering matters The label selector runs against the **index**; the text search runs against the matching streams **at query time**. That's the direct payoff of Loki's "index only labels" design — cheap storage doesn't mean slow queries *if your labels are chosen well*.

NOTE • The bridge Stage 3 converts **logs into a metric** (`count_over_time` over a `[5m]` range) — structurally a sibling of PromQL's `rate()`, and the bridge between the two signal types.

 **EXAM** LogQL = **stream selector (indexed)** → **line filters (query-time)** → **optional metric wrapper** · `|=` keep, `!=` drop · `count_over_time(...[5m])` turns error lines into a per-pod rate.

 The other half of the pair: → [Part 9-b: PromQL](#)

PromQL — Querying Prometheus

- **TASK [Core examples — annotated by source component]** know who exposes each metric

```
# Health of every scrape target (1 = up, 0 = down)
up

# Only the observability stack itself
up{namespace="observability"}

# Per-container CPU rate over 5m — metric from cAdvisor (via kubelet)
sum by (pod) (rate(container_cpu_usage_seconds_total[5m]))

# Container memory right now — cAdvisor
sum by (pod) (container_memory_usage_bytes)

# Node memory available, as a fraction — metric from node-exporter
node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes
```

- **TASK [Two mechanics worth internalizing]** range vectors & aggregation

[5m] = a range vector

Selects **many points over 5 minutes** — functions like `rate()` **require** a range. A single instant value can't produce a rate.

sum by (pod) = aggregation

Aggregates across labels, **keeping** `pod` as the grouping dimension — the same labels Loki uses, so results line up across both backends.

- **TASK [Instant vs trend debugging]** pick the right question

"Right now, is it broken?" → `up · raw stream {namespace="..."}`

"Trending toward broken?" → `rate() over [5m] · errors/pod · memory fraction graphs`

KEY IDEA Because both query languages select by the **same labels**, the same `sum by (pod)` grouping works on both sides — a per-pod error *rate* from Loki lines up exactly with a per-pod CPU *rate* from Prometheus in adjacent Grafana panels.

 **EXAM** `up` = scrape health · `rate(counter[5m])` needs a **range vector** · `sum by (pod)` keeps the grouping label · cAdvisor → container metrics · node-exporter → `node_*` metrics.

 Getting *your own app* into the picture: → [Part 10: App Integration](#)

Application Integration

TASK [Logs — zero configuration] it just works

- ✓ **LOGS** **Promtail automatically collects logs from every pod in the cluster** — no annotations, no CRDs, nothing. Its DaemonSet + Kubernetes service discovery pick up new pods as they appear. Deploy an app, and its logs (already labeled with namespace/pod/container) are in Loki.

TASK [Metrics — two requirements] opt-in by design

1. The app must **expose a `/metrics` endpoint on an `http` port** (client libraries exist for every major language, publishing counters/gauges/histograms in Prometheus format).
2. Create a **`ServiceMonitor` Custom Resource** whose label selector matches your application — Prometheus discovers it through the CRD (Helm/operator auto-wires the rest).

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: my-app
  namespace: observability
  labels:
    release: kube-prometheus-stack # must match what the Prometheus operator selects
spec:
  selector:
    matchLabels:
      app: my-app # selects YOUR app's Service by its labels
  namespaceSelector:
    matchNames:
      - default # where the app's Service lives
  endpoints:
    - port: http # the NAME of the Service port serving /metrics
      path: /metrics
      interval: 30s
```

selector.matchLabels

Finds the app's **Service**.

endpoints.port

Refers to the Service port **name** — hence "on an `http` port."

release label

What the operator's discovery matches on — the "**auto-wiring**" Helm provides.

- ✓ **VERIFY IT WORKED** Query `up{namespace="default"}` in Explore — your app should appear with value `1` within a scrape interval or two.

🎓 **EXAM** Logs = **zero config** (Promtail auto-collects) · Metrics = **`/metrics` endpoint + `ServiceMonitor` CRD** · `release:` label connects the CRD to the operator · Verify with `up{namespace="default"} == 1`.

🔗 When things go wrong — the playbook: → [Part 11: Troubleshooting](#)



Troubleshooting Playbook — Scenarios 1–3

Reason from **symptom** → **diagnosis** → **root cause** → **fix**. The diagnostic command sequence matters more than memorizing fixes.

● Scenario 1 — Pods stuck in Pending

Symptom	<code>kubectl get pods -n observability</code> → pods sitting in Pending , not starting
Diagnose	<code>kubectl describe pod <pod> -n observability</code> → events show FailedScheduling: Insufficient cpu / memory
Root cause	Cluster lacks free CPU/memory — the chart's default resource <i>requests</i> don't fit on the Minikube node, and the scheduler will not place a pod whose requests don't fit
Fix	LOWER REQUESTS (the <code>--set</code> overrides from Part 8) or GROW MINIKUBE (<code>--cpus=...</code> <code>--memory=...</code>)
Lesson	<code>Pending</code> = scheduling problem = look at <i>requests vs available</i> before anything else

● Scenario 2 — Grafana OOMKilled

Symptom	Grafana restarts repeatedly; <code>kubectl describe pod</code> shows OOMKilled (exit code 137)
Root cause	The container ran out of memory — it exceeded its own memory <i>limit</i>
Diagnose first	Check the node's free memory first (<code>kubectl top nodes</code>) — do not blindly raise the Grafana limit
Fix	Node has headroom → RAISE LIMIT via <code>--set</code> ; node exhausted → FREE CAPACITY (disable components, trim pods, grow Minikube)
Lesson	<code>OOMKilled</code> = limit problem; node exhaustion = capacity problem. Diagnose which one you have.

● Scenario 3 — Grafana CrashLoopBackOff with a provisioning error

Symptom	Grafana cycles CrashLoopBackOff ; logs show "Datasource provisioning error."
Diagnose	<code>kubectl logs <grafana-pod> -n observability</code> · <code>kubectl get configmap -n observability</code> → two datasource ConfigMaps, both marked default
Root cause	Two datasources marked default — Grafana permits exactly one; provisioning fails at startup every time → crash loop. <i>Not</i> a resource problem — a config validation failure can crash-loop a perfectly sized pod.
Fix	Prevent: install loki-stack with <code>defaultDatasourceEnabled=false</code> · Repair: remove <code>isDefault: true</code> from the extra ConfigMap, then <code>kubectl delete pod</code> (Deployment recreates it clean)
Lesson	Read the crash logs — the error names the subsystem (provisioning), which points straight at ConfigMaps. This is the star interview story.

Scenarios 4–5 + Cleanup

Scenario 4 — "Connection refused" from Windows / WSL

Symptom	Browser on Windows can't reach <code>localhost:3000</code> even though port-forward is running
Root cause	The forward was bound to loopback inside the WSL VM only — the <code>--address 0.0.0.0</code> flag was missing
Fix	RE-RUN FULL COMMAND <pre>kubectl port-forward --address 0.0.0.0 -n observability svc/kube-prometheus-stack-grafana 3000:80</pre>
Lesson	On WSL, your browser and your cluster live on different network boundaries — bind all interfaces so the port relay works

Scenario 5 — "Grafana shows no data" (general reasoning flow)

Work the chain **in order** — the boring causes are the common ones:

1. **Is the backend pod running?** → `kubectl get pods -n observability` (a crashed Loki or Prometheus means no queries can succeed).
2. **Is the datasource alive?** → Explore → run `up` (Prometheus) or `{namespace="observability"}` (Loki). If *Explore* works but a dashboard doesn't, the problem is the dashboard's query/time-range — not the stack.
3. **Missing app metrics specifically?** → no `/metrics` endpoint, or the ServiceMonitor selector doesn't match — check `up` for the target.
4. **Right namespace? Right time range?** → always worth a glance.

12 Cleanup — removing the entire stack

```
helm uninstall kube-prometheus-stack -n observability
helm uninstall loki-stack -n observability
kubectl delete namespace observability
```

PRACTICAL NOTE Deleting the namespace removes anything the charts left behind *inside it* — but the **CRDs that kube-prometheus-stack installed are cluster-scoped** and may remain after uninstall. Delete them explicitly if you want a spotless cluster.

EXAM WSL port-forward needs `--address 0.0.0.0` · "No data" chain: **backend pod** → **datasource (Explore)** → **app target** → **namespace/time-range** · Cleanup: **two helm uninstalls + delete namespace** (+ cluster-scoped CRDs by hand).

Now zoom out — the whole system on one page: → [Part 13: The Mental Model](#)



Quick Revision — The Terms Table

Every term in this book, one line each. If you can define all of these without looking — you're ready.

■ TASK [Signals + Logs]

Term	One-line definition
Observability	Understanding system state from its own data — without shipping new code to ask a new question
Log	Discrete, timestamped event record — "what happened, and what did it say"
Metric	Numeric measurement over time — "how much, how fast, how often"
Trace	Full journey of one request across services — "where did the time go"
Visualization	Dashboards turning raw signals into something a human can act on
Loki	Log aggregation backend; indexes labels only , stores compressed chunks
Promtail	DaemonSet tailing <code>/var/log/pods</code> on every node; pushes to Loki
Distributor	Validates & hashes incoming log streams
Ingestor	Batches streams into chunks and writes them
Store	Keeps chunks + index (object storage or disk)
Querier	Executes LogQL queries coming from Grafana
Log Stream	A unique combination of labels — every distinct label set is its own stream
LogQL	Loki's query language: stream selector → line filters → metric wrapper
Chunk	Compressed batch of log lines in object storage / disk

■ TASK [Metrics · Viz · Deploy]

Term	One-line definition
Prometheus	Pull-based metrics backend storing time series
Pull Model	Prometheus scrapes targets on an interval; dead target = <code>up 0</code>
Time Series	Metric name + key/value labels + timestamped values
PromQL	Prometheus query language — aggregation, filtering, rates
Alertmanager	Routes PromQL alert conditions to Slack/email/PagerDuty (<i>disabled here</i>)
node-exporter	DaemonSet exposing host-level metrics — the machine layer
kube-state-metrics	Watches the K8s API — object state: replicas, restarts
cAdvisor	Built into the kubelet — per-container CPU/memory usage
ServiceMonitor	CRD declaring which Services/Pods Prometheus scrapes
Grafana	Unified UI; stores nothing — queries data sources live
Data Source	URL + auth + query language (PromQL or LogQL) in Grafana
Dashboard / Panel	A dashboard is a collection of panels; each renders one query
Helm	K8s package manager: charts → releases from repositories
DaemonSet	One pod per node — for node-local collectors
port-forward	Tunnels a ClusterIP Service to <code>localhost</code> (<code>--address 0.0.0.0</code> on WSL)
Pending · OOMKilled · CrashLoopBackOff	The three failure states: scheduling / memory limit / startup crash



Concept Questions — with Distilled Answers

Q1 What is observability?

Understanding what's happening inside a system from the data it produces — **without shipping new code to ask a new question**. Pillars: logs, metrics, traces, visualization.

Q3 Why is Loki 10–20× cheaper than full-text systems?

It indexes **only metadata (labels)**, not log text; text is searched at query time. Chunks are compressed into **cheap object storage**.

Q5 Why are Promtail & node-exporter DaemonSets?

Their data is **node-local**: container logs live in each node's `/var/log/pods`; host metrics are per-machine. One pod per node = full coverage.

Q7 node-exporter vs kube-state-metrics vs cAdvisor?

Host-level machine metrics (DaemonSet) vs K8s *object state* from the API server (pod status, replicas, restarts) vs per-container usage built into the kubelet.

Q10 Why `helm upgrade --install` over `helm install`?
Idempotency — re-running **upgrades an existing release in place** instead of failing.

Q12 Why drop Prometheus retention to 6h in a lab?

TSDB storage grows with retention; a lab needs no history — 6h saves significant disk.

Q14 Natural next step for this stack?

Traces — deploy Grafana Tempo (or Jaeger/Zipkin), instrument apps with OpenTelemetry — completing all three signals.

Q2 Logs vs metrics vs traces — when each?

Logs = *what happened* (exact events). Metrics = *how much/how fast* (trends, alerting). Traces = *where the time went* (per hop). Debug flow: alert → trace → log.

Q4 Pull vs push — Prometheus vs Promtail?

Prometheus **pulls** (scrapes on an interval — central cadence control; dead target visible via `up=0`). Promtail **pushes** HTTP batches to Loki's distributor.

Q6 What makes up a Prometheus time series?

A **metric name + key/value labels + a sequence of timestamped values**.

Q8 What are ServiceMonitor/PodMonitor and why?

CRDs that **declaratively** tell Prometheus which Services/Pods to scrape — no manual IP lists; deploying the CRD is the config (Helm auto-wires it).

Q11 Why port-forward instead of NodePort/LoadBalancer?

Grafana is **ClusterIP** (internal only). Port-forward = zero-config lab access; NodePort/LB/Ingress expose it properly. On Windows/WSL you **must** pass `--address 0.0.0.0`.

Q13 How do logs and metrics link in this stack?

Same label model — Promtail attaches the exact labels Prometheus uses (namespace/pod/container/node), so a metric anomaly maps directly to that pod's log stream.

Q9 ★ Tell me about a bug you debugged in this stack.

Grafana `CrashLoopBackOff` with a datasource provisioning error: **both charts** provisioned a datasource marked **default**; Grafana allows exactly one. Diagnosed via pod logs + `kubectl get configmap`; fixed by `grafana.sidecar.datasources.defaultDataSourceEnabled=false` (or removing `isDefault: true`).

INTERVIEW TIP Q9 is **the star story** — it shows real debugging: read the crash log, form a hypothesis from the error's named subsystem (provisioning), verify with ConfigMaps, apply the targeted fix, and state the lesson (a config validation failure can crash-loop a perfectly sized pod).

Command Cheat Sheet

Command	What it does / tells you
<code>helm repo add prometheus-community https://prometheus-community.github.io/helm-charts</code>	Add chart repo
<code>helm repo add grafana https://grafana.github.io/helm-charts</code>	Add chart repo
<code>helm repo update</code>	Refresh repo indexes
<code>kubectl create namespace observability</code>	Dedicated namespace for the stack
<code>helm upgrade --install <release> <chart> -n observability --set ...</code>	Idempotent install/upgrade
<code>helm list -n observability</code>	Installed releases
<code>helm uninstall <release> -n observability</code>	Remove a release
<code>kubectl get pods -n observability</code>	First health check: Running / Pending / CrashLoopBackOff
<code>kubectl describe pod <pod> -n observability</code>	Events: FailedScheduling, OOMKilled, restart reason
<code>kubectl logs <pod> -n observability</code>	App-level errors (the provisioning error lives here)
<code>kubectl get configmap -n observability</code>	Find datasource provisioning ConfigMaps
<code>kubectl top nodes</code> / <code>kubectl top pods -n observability</code>	Actual CPU/memory vs requests
<code>kubectl delete pod <pod> -n observability</code>	Force a clean restart (e.g., after ConfigMap fix)
<code>kubectl port-forward --address 0.0.0.0 -n observability svc/kube-prometheus-stack-grafana 3000:80</code>	Access Grafana at <code>localhost:3000</code>

■ TASK [Query one-liners] memorize these four

```

{namespace="observability"}
up
sum by (pod) (rate(container_cpu_usage_seconds_total[5m]))
sum by (pod) (count_over_time({namespace="default"} |= "error" [5m]))

```

Loki – live streams from the namespace
 # Prometheus – 1 = scraped OK, 0 = down
 # CPU per pod (cAdvisor)
 # errors per pod (LogQL)

REMEMBER Diagnostic order that never fails: `get pods` → `describe pod` → `logs` → `get configmap` → `top nodes/pods`. Every scenario in Part 11 uses some subset of exactly these five.

Self-Quiz + Answer Key

1 Which Loki component validates and hashes incoming log streams?

2 From which path does Promtail read container logs, and as what workload type?

3 What three things make up a Prometheus time series?

4 Which component exposes per-container CPU/memory, and where does it run?

5 Why does Grafana crash-loop when two datasources are default, and what's the exact `--set` fix?

6 What does `up == 0` mean?

7 Why does Grafana itself store no data?

8 What are the three commands for full cleanup?

9 Which two community dashboard IDs cover Kubernetes logs and log volume?

10 Why must the port-forward use `--address 0.0.0.0` on Windows/WSL?

■ TASK [Answer key] check yourself honestly

1. The **Distributor** (then the Ingester batches streams into chunks; the Store keeps chunks + index).
2. `/var/log/pods` on its node; Promtail runs as a **DaemonSet** — one pod per node, because logs are node-local files.
3. **Metric name + key/value labels + a sequence of timestamped values.**
4. **cAdvisor**, built into the **kubelet**.
5. Grafana allows exactly **one default datasource**; provisioning fails at startup every restart. Fix: `--set grafana.sidecar.datasources.defaultDataSourceEnabled=false` when installing loki-stack.
6. A scrape target is **down** — Prometheus couldn't scrape it (visible failure is a pull-model benefit).
7. It's a **visualization layer**: it connects to data sources (Prometheus, Loki) and **queries them live**.
8. `helm uninstall kube-prometheus-stack -n observability` → `helm uninstall loki-stack -n observability` → `kubectl delete namespace observability`.
9. **13639** (Kubernetes Logs) and **12611** (Log volume), imported via the Grafana UI.
10. Without it, the forward binds to loopback inside WSL only, and the Windows browser gets **connection refused**; `0.0.0.0` binds all host interfaces.

FINAL CHECK If you can redraw the Part 13 architecture diagram from memory, define every term in Part 13-b without looking, answer all 14 interview questions out loud, and run the deployment from the cheat sheet — **you own this stack**. The route ends here. ►